checkout.php

```php
<?php
require _DIR_ . '/config.php';

function finish(string $title, string $message): never {
    echo '<!doctype html><html lang="en"><meta charset="utf-8"><title>' . htmlspecialchars($title) . '</title><body style="font:16px Arial;max-width:800px;margin:40px auto">';
    echo '<h1>' . htmlspecialchars($title) . '</h1><p>' . htmlspecialchars($message) . '</p><p><a href="index.php">Store</a> | <a href="cart.php">Cart</a> | <a href="orders.php">Orders</a></p></body></html>';
    exit;
}
if ($_SERVER['REQUEST_METHOD'] !== 'POST') { header('Location: cart.php'); exit; }

$customer = trim((string)($_POST['customer_name'] ?? ''));
$method = (string)($_POST['payment_method'] ?? 'CARD');
if ($customer === '') finish('Missing information', 'Enter a customer name.');
if (!in_array($method, ['CARD','EWALLET','BANK'], true)) $method = 'CARD';

$cart = $_SESSION['cart'] ?? [];
if (!$cart) finish('Empty cart', 'Add an item before checkout.');

$pdo = cart_db();
$ids = array_map('intval', array_keys($cart));
$marks = implode(',', array_fill(0, count($ids), '?'));
$stmt = $pdo->prepare("SELECT id,name,price,stock FROM products WHERE id IN ($marks)");
$stmt->execute($ids);
$products = $stmt->fetchAll();
$total = 0.0; $validItems = [];
foreach ($products as $p) {
    $qty = min((int)$cart[$p['id']], (int)$p['stock']);
    if ($qty > 0) {
        $total += $qty * (float)$p['price'];
        $validItems[] = ['product'=>$p,'qty'=>$qty];
    }
}
if (!$validItems || $total <= 0) finish('Checkout error', 'No valid items were found.');

/* Persist order before calling the independent payment service. */
$pdo->beginTransaction();
try {
    $stmt = $pdo->prepare("INSERT INTO orders (customer_name,total_amount,status) VALUES (?,?,'PENDING_PAYMENT')");
    $stmt->execute([$customer, number_format($total,2,'.','')]);
    $orderId = (int)$pdo->lastInsertId();
    $itemStmt = $pdo->prepare('INSERT INTO order_items (order_id,product_id,product_name,quantity,unit_price) VALUES (?,?,?,?,?)');
    foreach ($validItems as $i) {
        $itemStmt->execute([$orderId,$i['product']['id'],$i['product']['name'],$i['qty'],$i['product']['price']]);
    }
    $pdo->commit();
} catch (Throwable $e) {
    if ($pdo->inTransaction()) $pdo->rollBack();
    finish('Order error', 'Could not save the order.');
}
```

```php
$idempotencyKey = 'cart-order-' . $orderId;


$payload = [
    'order_id'        => $orderId,
    'amount'          => (float) $total,
    'customer_name'   => $customer,
    'payment_method'  => $method,
    'idempotency_key' => $idempotencyKey,
    'outcome'         => 'SUCCESS',   // change to FAILED / PENDING to test
];

$jsonPayload = json_encode($payload);
if ($jsonPayload === false) {
    $pdo->prepare('UPDATE orders SET status = ? WHERE id = ?')
        ->execute(['INTEGRATION_ERROR', $orderId]);
    finish('Integration error', 'Could not encode the payment request.');
}


$endpoint = 'http://localhost/payment-gateway/api/payments.php';

$ch = curl_init();
curl_setopt_array($ch, [
    CURLOPT_URL            => $endpoint,
    CURLOPT_POST           => true,
    CURLOPT_POSTFIELDS     => $jsonPayload,
    CURLOPT_HTTPHEADER     => [
        'Content-Type: application/json',
        'Accept: application/json',
    ],
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_TIMEOUT        => 15,
    CURLOPT_CONNECTTIMEOUT => 5,
]);

$rawResponse = curl_exec($ch);
$curlErrNo   = curl_errno($ch);
$curlErrMsg  = curl_error($ch);
$httpCode    = (int) curl_getinfo($ch, CURLINFO_HTTP_CODE);
curl_close($ch);


$newStatus     = 'INTEGRATION_ERROR';
$transactionId = null;
$userMessage   = 'Payment status could not be confirmed. Please check your order before attempting payment again.';

if ($curlErrNo !== 0) {
    $userMessage = 'The payment gateway did not respond in time. '
                 . 'Your payment may still be processing. '
                 . 'Please check your order status before retrying.';
}
elseif ($httpCode < 200 || $httpCode >= 300) {
```

checkout.php

```php
elseif ($httpCode < 200 || $httpCode >= 300) {
    $userMessage = "The payment gateway returned an error (HTTP {$httpCode}). "
                 . 'Your order has been flagged for review.';
}
else {
    $decoded = json_decode($rawResponse, true);

    if (!is_array($decoded) || json_last_error() !== JSON_ERROR_NONE) {
        $userMessage = 'The payment gateway returned an invalid response. '
                     . 'Your order has been flagged for review.';
    }
    else {
        $gatewayOrderId = filter_var($decoded['order_id'] ?? null, FILTER_VALIDATE_INT);
        $gatewayStatus  = strtoupper((string) ($decoded['status'] ?? ''));
        $gatewayTxnId   = isset($decoded['transaction_id'])
                        ? trim((string) $decoded['transaction_id'])
                        : '';

        if ($gatewayOrderId !== $orderId) {
            $userMessage = 'Payment gateway response did not match this order. '
                         . 'The order has been flagged for review.';
        }
        else {
            if ($gatewayTxnId !== '') {
                $transactionId = $gatewayTxnId;
            }

            switch ($gatewayStatus) {
                case 'SUCCESS':
                    $newStatus   = 'PAID';
                    $userMessage = 'Payment successful! '
                                 . ($transactionId ? "Transaction: {$transactionId}" : '');
                    break;

                case 'FAILED':
                    $newStatus   = 'PAYMENT_FAILED';
                    $userMessage = 'Payment failed. '
                                 . (string) ($decoded['message'] ?? 'Please try again.');
                    break;

                case 'PENDING':
                    $newStatus   = 'PAYMENT_PENDING';
                    $userMessage = 'Payment is pending confirmation. '
                                 . 'Please check your order status later.';
                    break;

                default:
                    $newStatus   = 'INTEGRATION_ERROR';
                    $userMessage = 'Unrecognized payment status. '
                                 . 'Your order has been flagged for review.';
                    break;
            }
        }
    }
}
```

```
    }
}


$update = $pdo->prepare(
    'UPDATE orders SET status = ?, gateway_transaction_id = ? WHERE id = ?'
);
$update->execute([$newStatus, $transactionId, $orderId]);

if ($newStatus === 'PAID') {
    unset($_SESSION['cart']);
}

finish('Checkout result', $userMessage);
```